

Q. Why do you feel we need to go above and beyond GPUs when it comes to AI processing?

A. The number of calculations you do in AI programs is very large. As it turns out, GPUs were better at running lots of math than regular CPUs. GPUs are actually built to run programs on pixels, which are independent. It was not a bad start and, obviously, people had real success with speeding that up.

If you actually look at the code for GPT3—when they trained it, they used five to ten thousand GPUs in a very large cluster. That must have cost something like a hundred million dollars! Also, the program itself is probably just a thousand lines of PyTorch. So, there are more GPUs than lines of code!

And some of the lines of code say something like, “Do a matrix multiply that’s 10,000 by 10,000”—that’s a very large amount of computation. To actually run that program on 10,000 GPUs is very complicated because the GPUs don’t just collaborate like 10,000 computers in one big thing. There are multiple layers. There might be about seven to ten layers of software depending on how you define it.

Hence, something different is needed here. For example, one of the things we at Tenstorrent like to do is—you write a thousand lines of code, we have a popular compiler that figures out how to break that problem up on a large number of processors. Our compiler can target from one to many chips. Right now, we’re working on the first 256 chips and we’re going to work our way up to 1000, which we think would be an interesting number for these kinds of training problems.

Q. What, according to you, is the right way to balance the power and performance of an AI chip?

A. Some AI models have very large sections of data. You would think making a really big RAM and putting

the processing next to it would work. The problem with that is that every time you want to read the data, it has to read across the big RAM, which is a high-power process.

So, the other way to do it is to take the data and break it into small pieces, and then put the processing next to the small piece. That’s how you get the power efficiency of having the data local to the processing, and not having to go so far across the chip—because a lot of power is used in moving data across the chip.

“Today the AI revolution is big and we’re going to bring something interesting with the combination of GPUs and AI processors.”

And you want the data and the processing to be local, but you also want enough data there to be interesting from a computing point of view. So, that’s one part. The other is that you want the data from one computation to go right to the next computation. You want to keep all the data on the chip and have it move through the pipeline without getting stuck, delayed, or written to memory. So, these two steps make the computation much more power-efficient.

Q. Most AI systems suffer from bottleneck problems. How can one create a perfect sync between data sharing and processing?

A. Keeping most of the data on-chip would solve this issue. The bottleneck is in the processing, and not

the memory. So, at the chip level, we can work around that bottleneck by keeping the data on-chip.

At the higher level, in the long run, this is going to be solved by reading data into AI models and having these AI models talk to each other, instead of re-reading lots of data over and over. Like when you learn a new thing, you don’t re-read all the stuff you’ve ever learned, right? You keep updating yourself. For example, when you add a word to a language model, it’s one word. You don’t add all the words you’ve ever learned. That’s a really interesting dynamic.

Q. What are your thoughts on open source software and hardware, especially in AI systems?

A. Intel, when they built their CPU, became the open hardware standard because they did a very good job of documenting, exposing their instruction set, and providing tools so everyone could use it. Way back, Intel architecture was built by seven different manufacturers. People were willing to write Assembly language programs for that.

Now on GPUs, the low-level instruction set is actually somewhat difficult to use, and the GPU vendors provide all the compiler software. You can write code at a high level and then compile it through the hardware. The GPU vendors actually change their instruction set almost every generation, so the user never sees the hardware directly. Tenstorrent is building the hardware and the software stack. Now, we’re going to open-source that software stack. So, people, if they want to, can go to the hardware level.

Q. Could you elaborate on a few concepts that have come up recently—like Software 2.0 and brain-like execution?

A. The big idea is, in Software 1.0 people write programs to do things.